

## 1. Minimum Number of Work Sessions to Finish the Tasks

There are  $n$  tasks assigned to you. The task times are represented as an integer array `tasks` of length  $n$ , where the  $i$ -th task takes `tasks[i]` hours to finish. A work session is when you work for at most `sessionTime` consecutive hours and then take a break.

You should finish the given tasks satisfying the following conditions:

- If you start a task in a work session, you must complete it in the same work session.
- You can start a new task immediately after finishing the previous one.
- You may complete the tasks in any order.

Given `tasks` and `sessionTime`, return the minimum number of work sessions needed to finish all the tasks.

*Note: `sessionTime` is always greater than or equal to the maximum element in `tasks[i]`.*

### Examples

#### Example 1

**Input:** `tasks = [1,2,3], sessionTime = 3`

**Output:** `2`

**Explanation:** Two work sessions needed.

- Session 1: tasks 1 and 2  $\Rightarrow 1 + 2 = 3$  hours.
- Session 2: task 3  $\Rightarrow 3$  hours.

#### Example 2

**Input:** `tasks = [3,1,3,1,1], sessionTime = 8`

**Output:** `2`

**Explanation:** Two work sessions needed.

- Session 1: all except last  $\Rightarrow 3+1+3+1 = 8$  hours.
- Session 2: last task  $\Rightarrow 1$  hour.

#### Example 3

**Input:** `tasks = [1,2,3,4,5], sessionTime = 15`

**Output:** `1`

**Explanation:** All tasks fit in one work session.

### Constraints

- $n == \text{tasks.length}$
- $1 \leq n \leq 14$

- $1 \leq \text{tasks}[i] \leq 10$
- $\max(\text{tasks}[i]) \leq \text{sessionTime} \leq 15$

## 2. Word Search

Given an  $m \times n$  grid of characters board and a string word, return true if word exists in the grid.

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

### Examples

#### Example 1

**Input:** board = `[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]]`  
word = `"ABCCED"`  
**Output:** `true`

|   |   |   |   |
|---|---|---|---|
| A | B | C | E |
| S | F | C | S |
| A | D | E | E |

#### Example 2

**Input:** board = `[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]]`  
word = `"SEE"`  
**Output:** `true`

|   |   |   |   |
|---|---|---|---|
| A | B | C | E |
| S | F | C | S |
| A | D | E | E |

#### Example 3

**Input:** board = `[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]]`  
word = `"ABCB"`  
**Output:** `false`

|   |   |   |   |
|---|---|---|---|
| A | B | C | E |
| S | F | C | S |
| A | D | E | E |

### Constraints

---

- `m == board.length`
- `n == board[i].length`
- `1 <= m, n <= 6`
- `1 <= word.length <= 15`
- board and word consists of only lowercase and uppercase English letters.

### 3. Grumpy Bookstore Owner

There is a bookstore owner that has a store open for  $n$  minutes. You are given an integer array `customers` of length  $n$  where `customers[i]` is the number of customers that enter the store at the start of the  $i$ -th minute, and all those customers leave after the end of that minute.

During certain minutes, the bookstore owner is grumpy. You are given a binary array `grumpy` where `grumpy[i]` is 1 if the owner is grumpy during the  $i$ -th minute, and 0 otherwise.

When the bookstore owner is grumpy, the customers entering during that minute are not satisfied. Otherwise, they are satisfied.

The bookstore owner knows a secret technique to remain not grumpy for  $minutes$  consecutive minutes, but this technique can only be used once.

Return the maximum number of customers that can be satisfied throughout the day.

#### Examples

##### Example 1

```
Input:  customers = [1,0,1,2,1,1,7,5]
        grumpy = [0,1,0,1,0,1,0,1], minutes = 3
Output: 16
Explanation: Owner stays not grumpy for last 3 minutes.
              Satisfied = 1+1+1+1+7+5 = 16.
```

##### Example 2

```
Input:  customers = [1], grumpy = [0], minutes = 1
Output: 1
```

#### Constraints

- $n == \text{customers.length} == \text{grumpy.length}$
- $1 \leq \text{minutes} \leq n \leq 2 \cdot 10^4$
- $0 \leq \text{customers}[i] \leq 1000$
- `grumpy[i]` is either 0 or 1.